TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico de Ciudad Cuauhtémoc

Ingeniería en Sistemas Computacionales

DESARROLLO DE APLICACIONES MÓVILES

Unidad 3 · Actividad 3

Fase de Proyecto: Arquitectura, Modelo de Datos, UI/UX y Prototipo Navegable

App móvil "Huerta Manzanos" · Sistema IoT para Monitoreo y Control de Riego en Huertas de Manzana

INTEGRANTES DEL EQUIPO

Axel Holguín Marrufo

Daniel Varela Legarreta

Reyes Fernando Zubía Arias

Grupo: 9° "B"

Docente: _____

Fecha de entrega: 09/11/2026

Índice

1. Documento de arquitectura 3
1.1 Descripción general del proyecto 3
1.2 Selección del tipo de arquitectura 4
1.3 Entorno y tecnologías 6
1.4 Modelo de datos y Diagrama ER 7
2. UI/UX: diseño visual y navegación 10
2.1 Wireframes de las pantallas principales 10
2.2 Guía de estilo 13
3. Prototipo navegable 15
4. Evidencias en el repositorio 18
5. Conclusiones 19
Referencias 19

1. Documento de arquitectura

Este documento describe la arquitectura técnica, el modelo de datos y el diseño de la interfaz de la aplicación móvil del proyecto integrador "Sistema IoT para Monitoreo y Control de Riego en Huertas de Manzana". La app es la interfaz con la que el encargado de la huerta consulta los sensores, controla el riego y revisa las alarmas que genera el sistema IoT (ESP32, MQTT y ThingsBoard) desarrollado en los avances anteriores.

1.1 Descripción general del proyecto

Problema que atiende la aplicación

En Ciudad Cuauhtémoc, Chihuahua, predominan las huertas de manzana familiares y de pequeña escala sin acceso a tecnologías de monitoreo. Los dueños riegan con técnicas tradicionales, por intuición o por recomendaciones, y los pocos que miden algo dependen de equipos rentados. Esto provoca un uso poco eficiente del agua, un recurso crítico en la región, como lo muestran los estudios de portainjertos de manzano bajo estrés hídrico.

El sistema IoT del proyecto ya obtiene la humedad de suelo de dos zonas, la temperatura y la humedad ambiente, y puede activar electroválvulas y una bomba. Lo que falta es una forma práctica de que el encargado vea esa información y controle el riego desde cualquier lugar. Además, debe quedar un registro de quién regó, cuándo, cuánto y qué alarmas ocurrieron.

Objetivo de la solución

Desarrollar una aplicación móvil que permita al encargado de la huerta:

- Monitorear en tiempo real la humedad de suelo por zona, la temperatura y la humedad ambiente.
- Controlar el riego de forma remota: regar o detener cada zona, cambiar entre modo automático y manual, y ajustar el umbral de humedad.
- Recibir y atender las alarmas que genera el Rule Engine de ThingsBoard (por ejemplo, "Humedad de suelo baja").
- Consultar el historial de lecturas, los eventos de riego y los accesos RFID.
- Registrar y administrar las tarjetas RFID de los encargados autorizados para el riego manual.

Público objetivo

Tabla 1. Usuarios de la aplicación

Usuario	Descripción	Necesidades principales
Propietario / administrador	Dueño de la huerta, con conocimientos técnicos básicos.	Ver el estado general, revisar el historial y consumo de riego, y dar de alta a los encargados y sus tarjetas RFID.
Encargado de riego	Trabajador que atiende la huerta día a día; usa el celular en campo, a veces con señal débil.	Saber qué zona necesita agua, regar con pocos toques y recibir alertas.

Por eso la interfaz prioriza textos grandes, pocos pasos por acción, confirmaciones claras y funcionamiento con conexión intermitente.

1.2 Selección del tipo de arquitectura

Decisión: aplicación híbrida (multiplataforma) desarrollada con Flutter, con una arquitectura interna por capas y un backend propio en la nube. Flutter genera, desde un solo código en Dart, apps que se compilan a código nativo ARM para Android e iOS. Por eso se clasifica como multiplataforma, pero tiene un rendimiento muy cercano al de una app nativa.

Tabla 2. Comparación de los tipos de arquitectura

Criterio	Nativa (Kotlin + Swift)	Web (PWA)	Híbrida (Flutter) ✔
Requerimientos	Cumple todo, pero hay que programar dos veces.	Notificaciones push limitadas en iOS; no funciona bien en segundo plano.	Cumple todo: push (FCM), WebSocket, caché offline y almacenamiento seguro.
Dispositivos	Una app por plataforma.	Cualquier navegador.	Android e iOS con un solo código; también web y escritorio para pruebas.
Recursos de desarrollo	El equipo tendría que dominar dos lenguajes.	HTML/CSS/JS.	Dart/Flutter, que el equipo ya domina por las unidades 1 y 2 de la materia.
Tiempo	Mayor (el doble de código y pruebas).	Menor.	Menor: hot reload y una sola base de código.
Despliegue	Dos tiendas, dos procesos de publicación.	Inmediato con hosting.	APK o Play Store desde el mismo proyecto; iOS cuando se requiera.
Rendimiento / UX	Excelente.	Depende del navegador.	Muy bueno (compilado AOT, 60 fps) con Material 3.

Justificación

- **Requerimientos:** la app necesita notificaciones push cuando hay una alarma, datos en tiempo real por WebSocket, caché local (sqflite) para cuando falla la señal en campo, y guardar el token de forma segura. Flutter cubre todo esto con paquetes oficiales y maduros. Una PWA no garantiza las notificaciones ni el trabajo en segundo plano en todos los teléfonos.
- **Dispositivos:** los encargados usan principalmente teléfonos Android de gama media, y el propietario podría usar iPhone. Con un solo código se atienden ambos.
- **Recursos:** el equipo es de 3 personas y ya trabajó con Flutter, sqflite y consumo de APIs REST en las unidades anteriores. Una app nativa duplicaría el trabajo.
- **Tiempo:** el proyecto se entrega en el semestre. El hot reload y los widgets de Material 3 aceleran el desarrollo de la interfaz.
- **Despliegue:** para el piloto basta con instalar un APK en los teléfonos de los encargados; después se puede publicar en Play Store sin cambiar de tecnología. Además, la app móvil oficial de ThingsBoard también está hecha en Flutter, lo que confirma que es compatible con la plataforma.

Diagrama de arquitectura

La aplicación sigue una arquitectura por capas. Cada capa solo se comunica con la de abajo, lo que facilita probar y cambiar partes sin afectar a las demás:

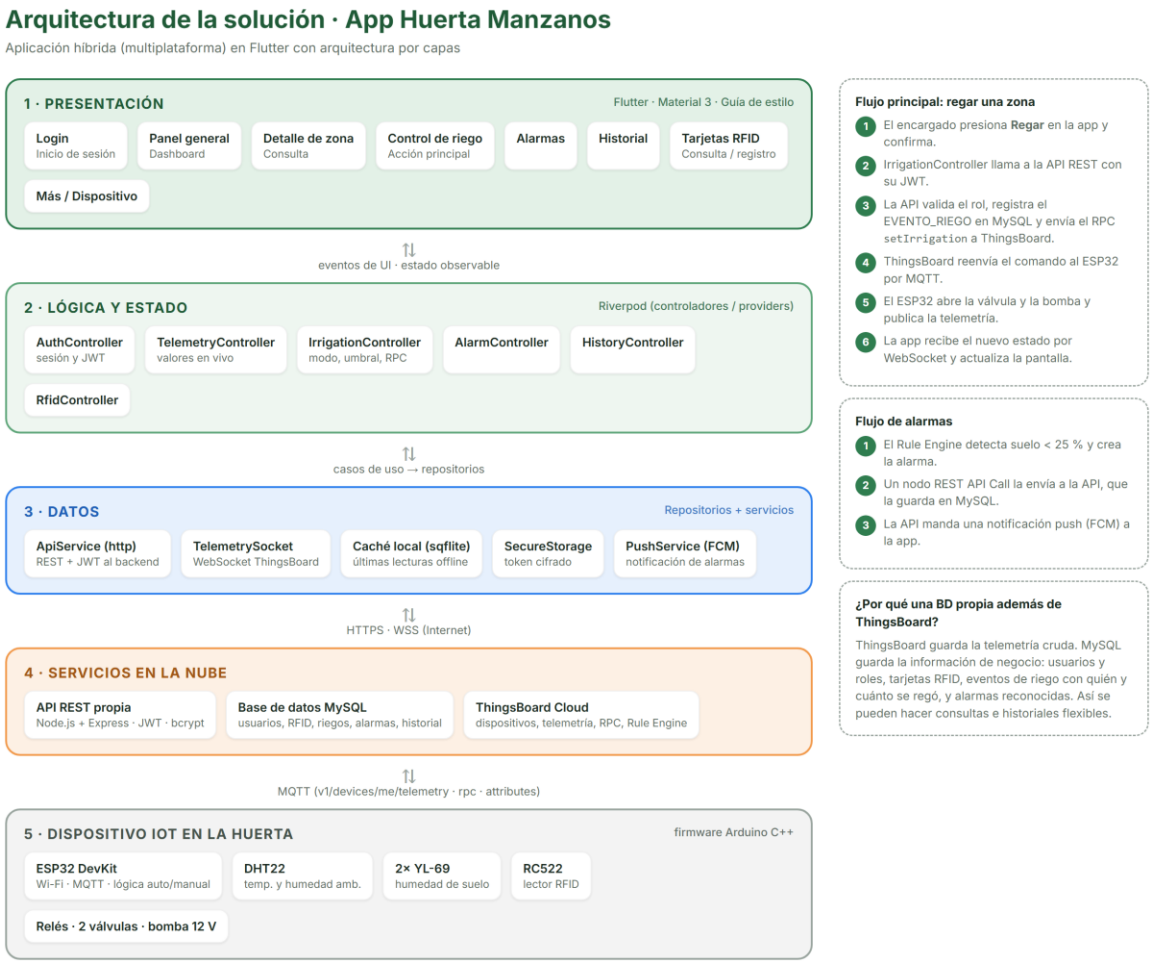


Figura 1. Arquitectura de la solución por capas y flujos principales.

Tabla 3. Capas de la arquitectura

Capa	Responsabilidad	Componentes
1. Presentación	Dibujar las pantallas y capturar las acciones del usuario. No contiene lógica de negocio.	Widgets de Flutter de las 10 pantallas, tema de la guía de estilo y go_router para la navegación.
2. Lógica y estado	Mantener el estado de la app y aplicar las reglas (validar, confirmar, decidir qué llamar).	Controladores de Riverpod: Auth, Telemetry, Irrigation, Alarm, History, Rfid.
3. Datos	Obtener y guardar datos sin que la UI sepa de dónde vienen.	ApiService (http + JWT), TelemetrySocket (WebSocket), caché sqflite, flutter_secure_storage, FCM.
4. Servicios en la nube	Autenticación, reglas de negocio, persistencia y comunicación con el IoT.	API REST en Node.js + Express, MySQL y ThingsBoard Cloud (RPC, telemetría, Rule Engine).

Capa	Responsabilidad	Componentes
5. Dispositivo IoT	Leer sensores y accionar el riego en la huerta.	ESP32 con DHT22, 2 sensores YL-69, lector RC522, relés, 2 electroválvulas y bomba.

Flujo principal: regar una zona desde la app

1. El encargado presiona "Regar" en la pantalla Control de riego y confirma en el diálogo.
2. IrrigationController envía `POST /api/zonas/{id}/riego` a la API con su token JWT y la duración elegida.
3. La API valida el rol del usuario, registra el EVENTO_RIEGO en MySQL y envía el RPC `setIrrigation {"zone":1,"state":true,"duration_s":60}` a ThingsBoard.
4. ThingsBoard entrega el comando al ESP32 por MQTT, en el tema `v1/devices/me/rpc/request/{id}`.
5. El ESP32 abre la electroválvula, enciende la bomba, responde el RPC y publica la telemetría con valve1 = true.
6. La app recibe la nueva telemetría por WebSocket y cambia el estado de la zona a "Regando".

Manejo de conexión intermitente: si no hay señal, la app muestra los últimos datos guardados en caché con la hora de actualización y desactiva los botones de riego, para no enviar comandos que no se pueden confirmar. Lo mismo hace el riego automático: depende del ESP32 y no de la app, así que sigue funcionando aunque el teléfono esté desconectado.

1.3 Entorno y tecnologías

Tabla 4. Entorno y tecnologías

Aspecto	Tecnología seleccionada
Lenguaje / framework	Dart 3.13 con Flutter 3.47 (Material 3)
Entorno de desarrollo	Visual Studio Code con las extensiones de Flutter y Dart; Android Studio (SDK 36 y emulador); dispositivo Android físico para pruebas en campo
Control de versiones	Git + GitHub (repositorio riego-huerta-app); GitHub Pages para publicar el prototipo
Diseño UI/UX	Prototipo navegable en HTML/CSS/JS con los tokens de la guía de estilo; wireframes generados del mismo prototipo en modo baja fidelidad
Backend	Node.js 24 + Express, jsonwebtoken (JWT), bcrypt, mysql2
Base de datos	MySQL 8 (datos de negocio) + telemetría cruda en ThingsBoard Cloud
Plataforma IoT	ThingsBoard Cloud: dispositivos, dashboard, RPC, atributos compartidos y Rule Engine
Firmware	Arduino C++ en ESP32 (PubSubClient, ArduinoJson, DHT, MFRC522)

Tabla 5. Dependencias principales de la app

Paquete Flutter	Uso en la app
flutter_riverpod	Manejo de estado (controladores por pantalla)
go_router	Navegación declarativa entre pantallas y barra inferior

Paquete Flutter	Uso en la app
http	Llamadas a la API REST con el token JWT
web_socket_channel	Telemetría en tiempo real desde ThingsBoard
sqflite	Caché local de lecturas para uso sin conexión
flutter_secure_storage	Guardar el token JWT cifrado
fl_chart	Gráficas de humedad de suelo en Historial
firebase_messaging	Notificaciones push de alarmas
intl	Formato de fechas y números en español

1.4 Modelo de datos y Diagrama ER

ThingsBoard guarda la telemetría cruda del sensor. La app necesita además información de negocio que no es telemetría: usuarios y roles, tarjetas RFID con las zonas que pueden regar, quién activó cada riego y con qué resultado, y qué alarmas se atendieron. Esta información se guarda en una base de datos MySQL propia, como se planteó en la justificación del proyecto.

Entidades

Tabla 6. Entidades del modelo de datos

Entidad	Descripción	Campos principales
USUARIO	Persona que usa la app (admin o encargado).	id_usuario (PK), nombre, email (UK), password_hash, rol, activo, creado_en
HUERTA	Huerta registrada y su propietario.	id_huerta (PK), id_propietario (FK), nombre, ubicacion, superficie_m2
DISPOSITIVO	Nodo ESP32 vinculado a ThingsBoard.	id_dispositivo (PK), id_huerta (FK), nombre, tb_device_id (UK), firmware, modo, estado, ultimo_contacto
ZONA	Zona de riego: una electroválvula y un sensor.	id_zona (PK), id_dispositivo (FK), numero, nombre, umbral_humedad_pct, valvula_abierta
LECTURA	Lectura periódica (cada 10 s) del nodo.	id_lectura (PK), id_dispositivo (FK), fecha_hora, temp_c, hum_amb
LECTURA_SUELO	Humedad de cada zona dentro de una lectura.	id_lectura (PK, FK), id_zona (PK, FK), humedad_pct
TARJETA_RFID	Tarjeta de un encargado.	id_tarjeta (PK), id_usuario (FK), uid (UK), activa, fecha_alta
TARJETA_ZONA	Zonas que puede regar cada tarjeta (tabla puente N:M).	id_tarjeta (PK, FK), id_zona (PK, FK)
ACCESO_RFID	Cada lectura de tarjeta en el nodo, válida o denegada.	id_acceso (PK), id_dispositivo (FK), id_tarjeta (FK, nulo), uid_leido, autorizado, accion, fecha_hora
EVENTO_RIEGO	Cada activación de riego y su resultado.	id_evento (PK), id_zona (FK), id_usuario (FK, nulo), id_acceso (FK, nulo), origen, inicio, fin, duracion_s, humedad_inicial, humedad_final

Entidad	Descripción	Campos principales
ALARMA	Alarma del Rule Engine y su atención.	id_alarma (PK), id_dispositivo (FK), id_zona (FK), tipo, severidad, estado, valor, creada_en, reconocida_por (FK), resuelta_en

Relaciones y cardinalidades

Tabla 7. Relaciones del modelo

Relación	Cardinalidad	Significado
USUARIO — HUERTA	1 : 0..N	Un usuario puede ser propietario de varias huertas.
HUERTA — DISPOSITIVO	1 : 1..N	Una huerta tiene al menos un nodo ESP32.
DISPOSITIVO — ZONA	1 : 1..N	Cada nodo controla sus zonas (2 en el prototipo).
DISPOSITIVO — LECTURA	1 : 0..N	El nodo registra una lectura cada 10 s.
LECTURA — LECTURA_SUELO — ZONA	1 : 1..N / N : 1	Cada lectura incluye la humedad de cada zona.
USUARIO — TARJETA_RFID	1 : 0..N	Un encargado puede tener una o más tarjetas.
TARJETA_RFID — ZONA (vía TARJETA_ZONA)	N : M	Una tarjeta puede regar varias zonas y una zona puede regarse con varias tarjetas.
DISPOSITIVO — ACCESO_RFID	1 : 0..N	El lector del nodo registra cada tarjeta acercada.
TARJETA_RFID — ACCESO_RFID	0..1 : 0..N	Si el UID no está registrado, el acceso queda sin tarjeta (denegado).
ZONA — EVENTO_RIEGO	1 : 0..N	Cada zona acumula su historial de riegos.
USUARIO — EVENTO_RIEGO	0..1 : 0..N	El riego automático no tiene usuario; el manual sí.
ACCESO_RFID — EVENTO_RIEGO	0..1 : 0..1	Un acceso autorizado origina como máximo un riego.
DISPOSITIVO / ZONA — ALARMA	1 : 0..N / 0..1 : 0..N	Las alarmas se asocian al nodo y, si aplica, a la zona.
USUARIO — ALARMA (reconoce)	0..1 : 0..N	Registra qué usuario atendió cada alarma.

Diagrama Entidad-Relación

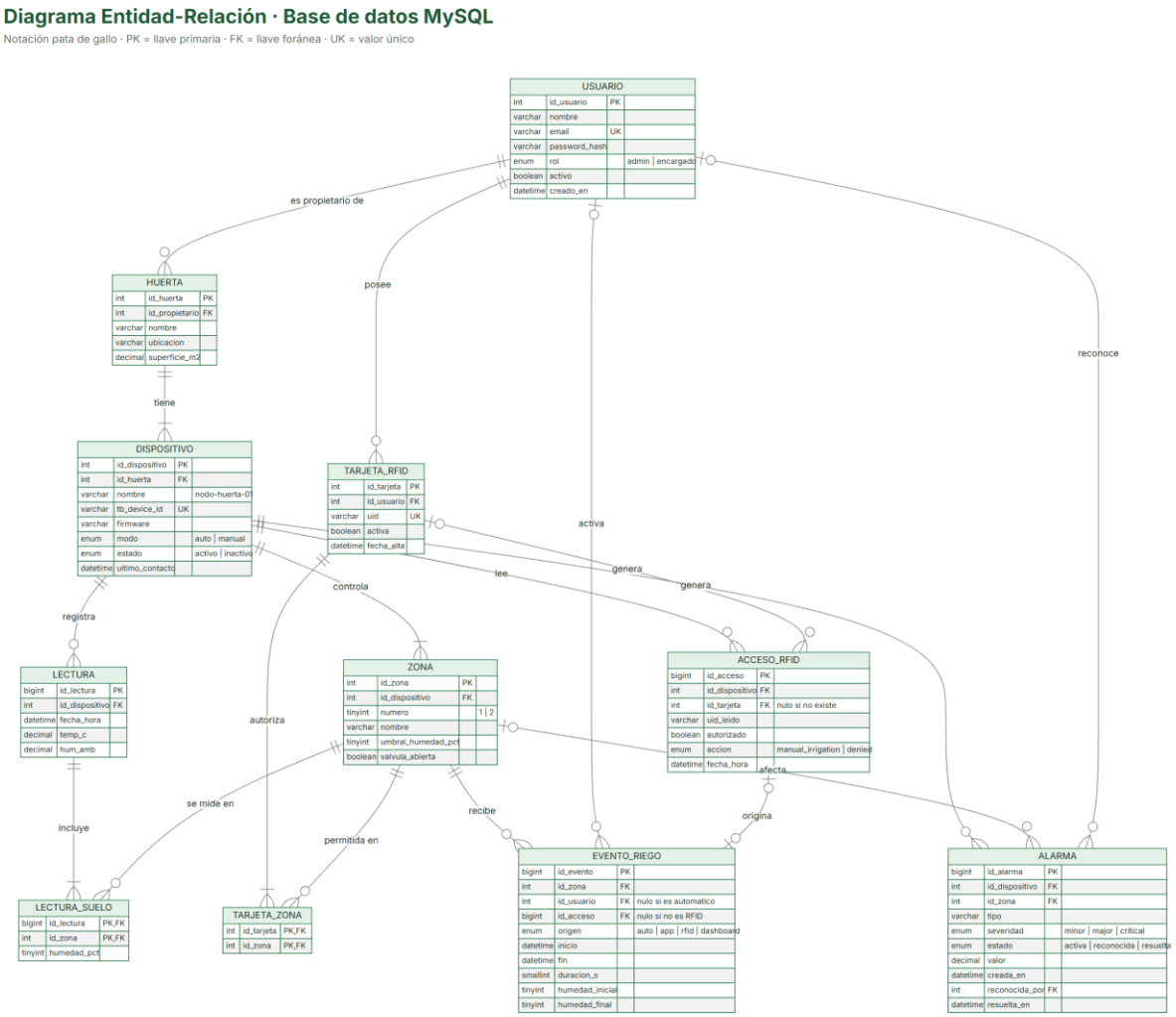


Figura 2. Diagrama ER de la base de datos MySQL (notación pata de gallo).

El script de creación `docs/modelo_datos.sql` se ejecutó en MySQL 8.0.30: crea las 11 tablas con sus llaves primarias, foráneas, restricciones UNIQUE y CHECK, e inserta datos de ejemplo. Las decisiones principales del modelo son:

- **Normalización:** la humedad de suelo se separa en LECTURA_SUELO, así se pueden agregar más zonas sin modificar la tabla LECTURA.
- **N:M con tabla puente:** TARJETA_ZONA permite que cada encargado tenga permisos solo en ciertas zonas, como se ve en la pantalla Registrar tarjeta.
- **Llaves foráneas opcionales:** id_usuario e id_acceso en EVENTO_RIEGO aceptan nulo para distinguir el riego automático, el de la app y el de RFID, sin duplicar tablas.
- **Índices:** se agregaron índices por dispositivo y fecha para que las consultas de Historial (1 h, 24 h, 7 d) sean rápidas.

2. UI/UX: diseño visual y navegación

El diseño parte de los mockups del segundo avance del proyecto IoT (Panel general, Control de riego, Alarmas e Historial). Para esta fase se agregaron las pantallas de inicio de sesión, detalle de zona (consulta), tarjetas RFID con su formulario de registro, más opciones y dispositivo. Se definieron los wireframes de baja fidelidad, la navegación completa y una guía de estilo.

2.1 Wireframes de las pantallas principales

Mapa de navegación

La app usa una barra de navegación inferior con 5 pestañas (Panel, Riego, Alarmas, Historial y Más) que está disponible en todas las pantallas principales. Las pantallas secundarias se abren desde botones y tienen una flecha para regresar.

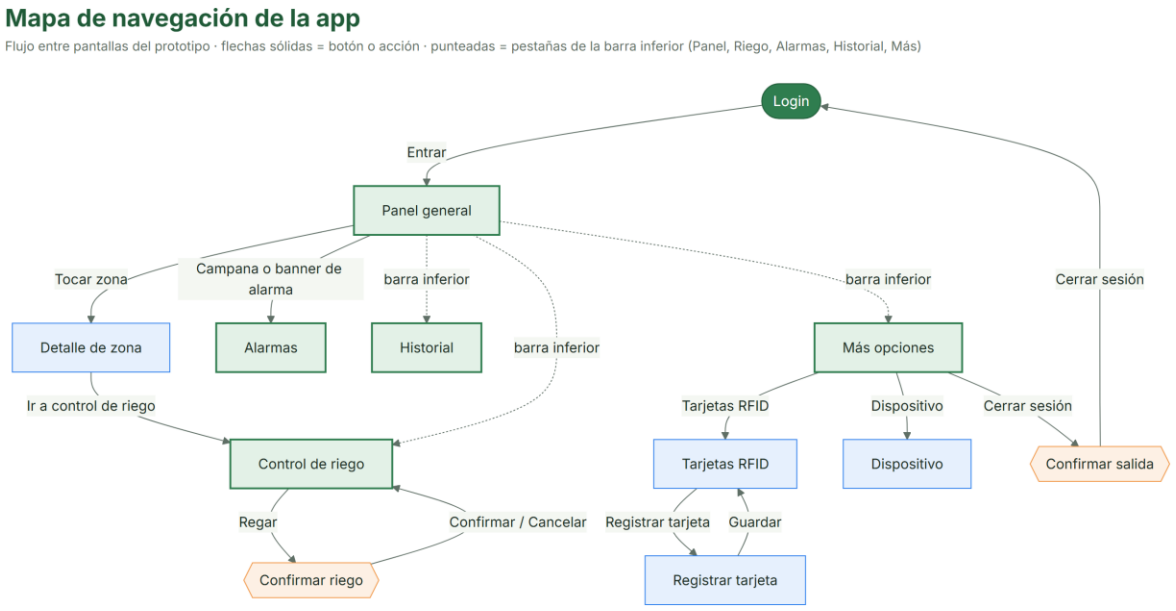


Figura 3. Mapa de navegación entre pantallas.

Wireframes

Los wireframes muestran la distribución de controles y la jerarquía visual en escala de grises, sin colores finales. Las gráficas se representan con un recuadro marcado.



Figura 4. Login



Figura 5. Panel general (dashboard)



Figura 6. Detalle de zona (consulta)



Figura 7. Control de riego (acción principal)



Figura 8. Alarmas



Figura 9. Historial (consulta)



Figura 10. Más opciones

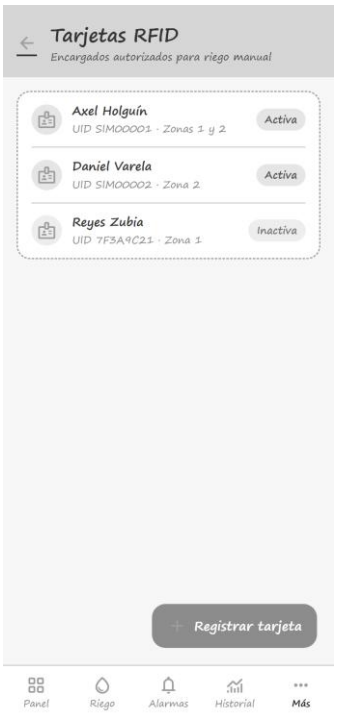


Figura 11. Tarjetas RFID (consulta)



Figura 12. Registrar tarjeta (registro)

Tabla 8. Descripción de los wireframes

Pantalla	Controles y jerarquía visual	Navegación
Login	Logo y nombre arriba; card con campos de correo y contraseña (con ícono y botón para mostrarla); botón principal "Entrar" a todo lo ancho; enlace de recuperación.	Entrar → Panel general.
Panel general	Header con el nodo y su estado en línea, y la campana con contador. Arriba los valores más importantes en grande (temperatura y humedad ambiente, grid de 2), después la card de humedad por zona con barras y chips, el estado de la bomba y un banner de alarma.	Tocar zona → Detalle; campana o banner → Alarmas; barra inferior.
Detalle de zona	Anillo con la humedad actual, chip de estado, umbral y válvula en grid de 2, lista de últimos riegos y botón principal al final.	Botón → Control de riego; flecha → Panel.
Control de riego	Control segmentado de modo, slider de umbral, pills de duración y una card por zona con los botones Regar (primario) y Detener (destructivo).	Regar → diálogo de confirmación → mensaje de resultado.
Alarmas	Filtros (Activas, Todas, Resueltas); cards con borde de color según el estado, chip de severidad y botón Reconocer.	Reconocer actualiza la card; barra inferior.
Historial	Filtros de rango (1 h, 24 h, 7 d), gráfica de humedad por zona con la línea del umbral, lista de accesos RFID y de eventos de riego.	Barra inferior.
Más opciones	Card del usuario; menú con Tarjetas RFID, Dispositivo y notificaciones; Cerrar sesión separado en rojo.	Tarjetas → Tarjetas RFID; Dispositivo; Cerrar sesión → Login.
Tarjetas RFID	Lista de encargados con UID, zonas y chip Activa/Inactiva; botón flotante "Registrar tarjeta".	Botón → Registrar tarjeta; flecha → Más.

Pantalla	Controles y jerarquía visual	Navegación
Registrar tarjeta	Formulario con nombre, UID (se lee desde el nodo), zonas permitidas y botón Guardar; validación con mensajes de error.	Guardar → Tarjetas RFID con mensaje de confirmación.

2.2 Guía de estilo

La guía completa está en `docs/Guia_de_Estilo.pdf`. Sus valores se definieron como tokens (variables CSS en el prototipo y `ThemeData` en Flutter) para aplicarlos igual en toda la app. Este es un resumen:

Tabla 9. Paleta de colores

Rol	Color	Hex	Uso
Primario	Verde huerta	#2F7D4F	Header, botón principal, pestaña activa
Primario oscuro / claro	Verde	#1F5A37 / #E3F1E7	Presionado, títulos / chips y foco
Secundario	Azul agua	#2F80ED	Riego, información, Zona 1 en gráficas
Acento	Rojo manzana	#D64545	Detener, errores, cerrar sesión
Advertencia	Ámbar	#F2994A	Humedad baja, alarmas activas
Éxito	Verde	#27AE60	Confirmaciones, acceso RFID válido
Neutros	Fondo / superficie / texto	#F4F7F2 · #FFFFFF · #1C2B22 · #5F6F66	Fondos, cards y textos

Tabla 10. Tipografía Inter

Estilo	Tamaño	Peso	Uso
Display	28 px	Bold	Valores de sensores (26 °C, 43 %)
Título de pantalla	22 px	Bold	Header
Título de card	16 px	SemiBold	Secciones
Cuerpo / botón	15 px	Regular / SemiBold	Texto general y botones
Etiqueta	12 px	Regular	Fechas, ayudas, unidades
Navegación	11 px	Medium / Bold	Barra inferior

- **Iconografía:** Material Symbols Rounded de 24 px (20 px dentro de inputs), con contorno; rellenos solo en la pestaña activa. Dentro de listas van en círculos de 36 px con el fondo del color de estado.
- **Botones:** alto de 48 px y radio de 12 px. Tipos: primario (relleno verde, uno por pantalla), secundario (borde verde), destructivo (borde rojo), texto y deshabilitado (45 % de opacidad).
- **Espaciado:** retícula de 4 px. Margen de pantalla de 16 px, 12 px entre cards y 8 px entre elementos relacionados.
- **Bordes y sombras:** cards con radio de 16 px y sombra suave (nivel 1); diálogos con radio de 20 px y sombra nivel 2; chips con radio total.

- **Layout:** diseño base de 390 × 844 px, header verde, contenido en una columna y barra inferior de 64 px. Las acciones quedan abajo, al alcance del pulgar.
- **Accesibilidad:** contraste del texto blanco sobre el verde primario de 5.0:1 y del texto principal de 13.7:1 (WCAG AA). Los estados siempre se indican con texto o ícono además del color.

3. Prototipo navegable

El prototipo está construido en HTML, CSS y JavaScript con los mismos tokens de la guía de estilo. Se ve dentro de un marco de teléfono en la computadora y en pantalla completa en el celular. Está publicado en GitHub Pages:

https://axelholguin327-ux.github.io/riego-huerta-app/

Todos los botones están enlazados a su pantalla. Como aún no hay conexión con la base de datos, las acciones se simulan con datos de ejemplo, diálogos de confirmación y mensajes que indican qué comando se enviaría al sistema, por ejemplo "Zona 1: riego iniciado por 60 s (setIrrigation)".

Tabla 11. Flujos cubiertos por el prototipo

Flujo	Recorrido en el prototipo
Pantalla inicial	Login: si los campos están vacíos o el correo no es válido, marca los errores en rojo. Si son correctos, muestra "Validando…" y entra al Panel general.
Pantalla principal	Panel general con valores, zonas, bomba y alarma activa; todas las cards son interactivas.
Consulta	Panel → tocar Zona 2 → Detalle de zona. Historial: cambiar el rango (1 h, 24 h, 7 d) redibuja la gráfica.
Acción principal (riego)	Riego → cambiar modo, mover el umbral, elegir la duración → Regar → diálogo de confirmación → chip "Enviando…" → "Regando · 60 s" y mensaje de confirmación. Detener cierra la válvula.
Atender alarma	Campana o banner → Alarmas → Reconocer: la card cambia a "Reconocida" y desaparece el contador de la campana.
Registro	Más → Tarjetas RFID → Registrar tarjeta → Leer tarjeta (simula el lector RC522) → Guardar: valida los campos y agrega la tarjeta a la lista.
Salir	Más → Cerrar sesión → diálogo de confirmación → Login.

Capturas del prototipo

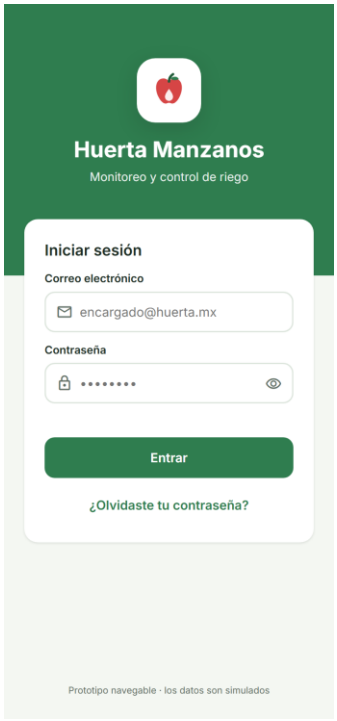


Figura 13. Login

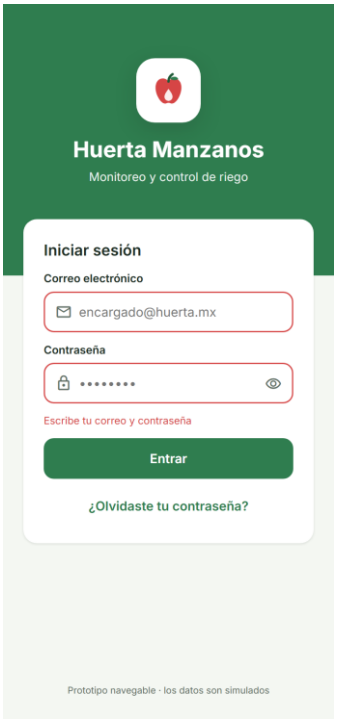


Figura 14. Validación del login



Figura 15. Panel general



Figura 16. Detalle de zona



Figura 17. Control de riego

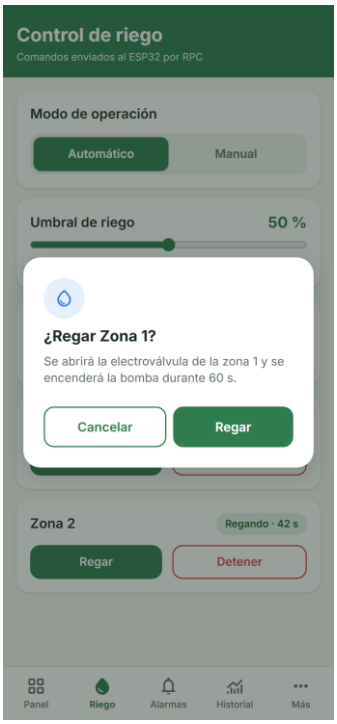


Figura 18. Confirmación de riego



Figura 19. Alarmas

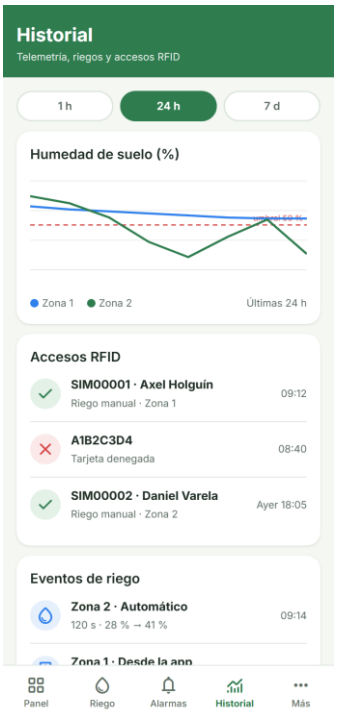


Figura 20. Historial

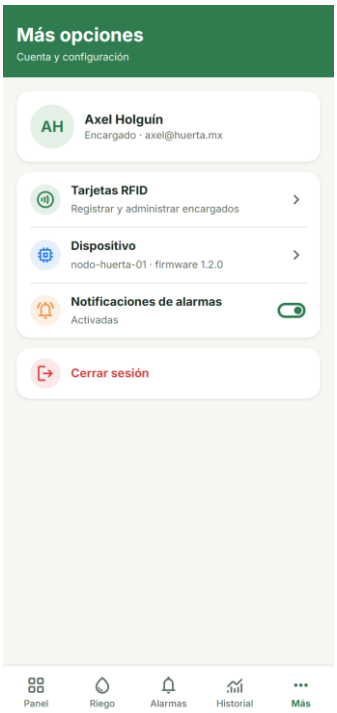


Figura 21. Más opciones

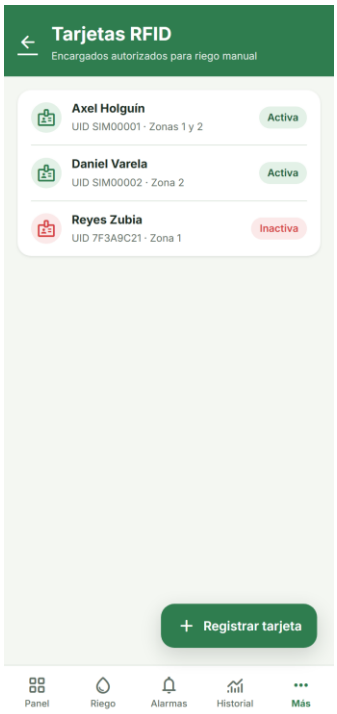


Figura 22. Tarjetas RFID

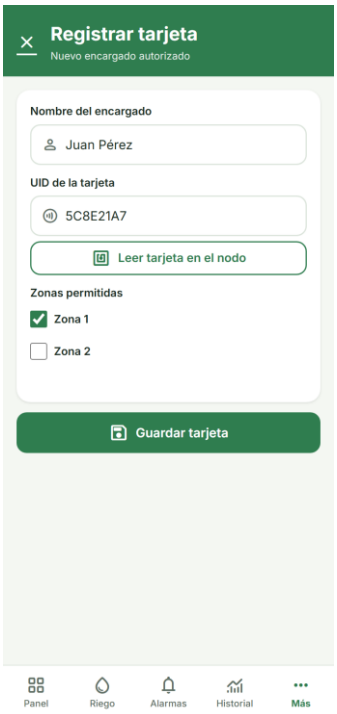


Figura 23. Registrar tarjeta

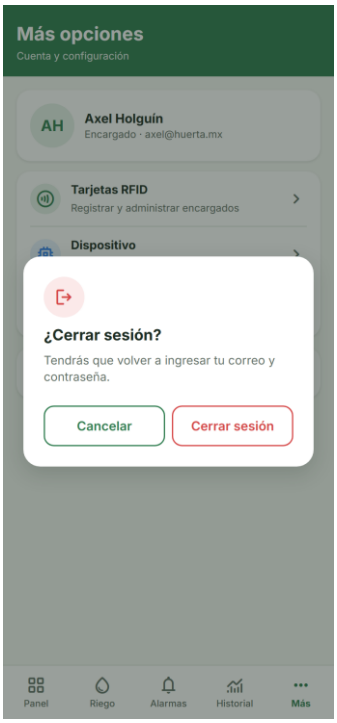


Figura 24. Cerrar sesión

4. Evidencias en el repositorio

Repositorio del proyecto:

https://github.com/axelholguin327-ux/riego-huerta-app

```
riego-huerta-app/
├─ README.md                      Descripción y enlaces
├─ index.html                     Redirige al prototipo (GitHub Pages)
├─ docs/
│   ├─ Documento_Arquitectura.pdf  Este documento
│   ├─ Guia_de_Estilo.pdf          Guía de estilo completa
│   ├─ diagrama_ER.png             Diagrama entidad-relación
│   ├─ diagrama_arquitectura.png   Diagrama de capas
│   └─ modelo_datos.sql            Script MySQL del modelo
└─ design/
    ├─ prototipo/index.html        Fuente del prototipo navegable
    ├─ guia-estilo/guia_estilo.html Fuente de la guía de estilo
    ├─ diagramas/                  Fuente y PNG de ER, arquitectura y navegación
    ├─ wireframes/                 Capturas de los wireframes
    ├─ capturas-prototipo/         Capturas del prototipo
    └─ capturar.ps1                Script que regenera las capturas
```

Los archivos fuente de diseño son el HTML del prototipo y de la guía de estilo; los wireframes se obtienen del mismo prototipo en modo baja fidelidad (agregando ?modo=wireframe a la dirección). Los commits se hicieron por etapa de esta fase:

Tabla 12. Historial de commits de la fase

Commit	Contenido
docs: agregar modelo de datos MySQL y diagrama ER	Script SQL probado y diagrama entidad-relación
docs: agregar diagrama de arquitectura por capas	Diagrama de capas y flujos
design: definir guía de estilo (colores, tipografía, componentes)	Fuente HTML y PDF de la guía
design: crear prototipo navegable de la app	10 pantallas enlazadas, diálogos y mensajes simulados
design: agregar wireframes, capturas y mapa de navegación	Imágenes de wireframes, prototipo y navegación
docs: agregar documento de arquitectura y README	Documento PDF final y README con enlaces

5. Conclusiones

En esta fase se definió cómo será la aplicación móvil del sistema de riego antes de programarla. La arquitectura híbrida con Flutter permite atender Android e iOS con un solo código, aprovechando lo que el equipo ya domina. La separación por capas hace que la app sea fácil de probar y de conectar después con la API y con ThingsBoard.

El modelo de datos complementa a ThingsBoard con la información de negocio que el proyecto necesita: quién riega, con qué tarjeta, cuánta agua se aplicó y qué alarmas se atendieron. El script se probó en MySQL para confirmar que las relaciones son correctas.

Por último, los wireframes, la guía de estilo y el prototipo navegable permitieron validar el flujo completo del usuario, desde el inicio de sesión hasta regar una zona, atender una alarma o registrar una tarjeta. El diseño es consistente, con pocos pasos y pensado para usarse en campo. Lo siguiente es implementar estas pantallas en Flutter y conectarlas con el backend.

Referencias

Flutter. (2026). Documentación oficial de Flutter. https://docs.flutter.dev

Google. (2026). Material Design 3. https://m3.material.io

ThingsBoard. (2026). ThingsBoard Cloud documentation: MQTT API, RPC y Rule Engine. https://thingsboard.io/docs

Oracle. (2026). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

Holguín, A., Varela, D. y Zubía, R. (2026). Sistema IoT para Monitoreo y Control de Riego en Huertas de Manzana: 2º avance. Instituto Tecnológico de Ciudad Cuauhtémoc.